coggle
made for free at coggle.it

- Sender
  - Socket A
    - Connect
    - Flag -> Ready
    - Send Message
    - Flag -> Seen
    - Read Message
    - Read Response
- Receiver
  - Socket B
    - Acccept
    - Flag -> Read
    - Flag -> Done

- Pipe(File)
  - First Byte
    - Locked for check if Can use the system or no, controlled by host.
  - Socket
    - Socket is 1024 Byte, Allocated for each Pear
    - Byte 1 to 16 is for This Socket ID
      - Part 1(1 - 16 byte)
    - For State (Active or Inactive)
      - Byte 17
      - Part 2(17 - 64 byte)
    - For Flags (Ready,Read ,Seen, Done)
      - Byte 18
    - For Errors (True and False)
      - Byte 19
    - Set Peer Socket ID (Read by host Socket)
      - Byte 20 - 36
    - Connection Reason (Under 25 char string)
      - Byte 37 - 62
    - If Rooms or Outside File Will use, Room ID/File path will set on Host Pocket At first.
      - Use Pocket, Rooms or Outside file for reading 0->Pocket; 1->Room; 2->File;
      - Byte 63
    - For Accept or Reject Connection
      - Byte 64
    - Socket Pocket
      - Part 3 (65 - 1024 Byte)
  - Pocket
    - 480 Byte to each Peer for Small Data Transfering & Connection Setting
    - Can use for multi-Reading in Large Data Transfering, Divide Data for Parts and Set Data in Room one by one while stay Connection between Peers on Pocket.
    - If Multi-Transfering On, Host Pocket Decide when to start, stream and stop
    - First byte in Pocket tell peer if this line read or no. (Reading Flag)
      - Byte is 0 (Nothing to Read)
      - Host put data
      - Host flip byte make it 1(New Data)
      - Peer flip byte to 2 (Read and ready to New)
      - Host flip byte to 0 (Nothing to Read)
  - Room
    - Room ID
      - Part 1 (1 - 16 Byte)
    - Room Size
      - Part 2 (17 - 32 byte)
    - Reading Flag
      - Part 3 (Byte 33)
    - Store Data
      - Part 4 (Byte 34 to Rest of Room...)
    - Place to Store Data(outside the Socket)
    - Size is Flexible ,But with fixed Number(1kb, 4kb, 2mb, 16mb... and so on)
    - Minimal Size for each Room is 1024 byte (1 Kilo Byte)
- Deep Dive
- Mapping File
  - Points Sockets/Rooms IDs to it position
  - On init, Entire File Read and Saved in Ram
  - Uses "Scanner" Tool in its Architecture (Simple)
  - Depends on fstream, string based.
  - If add, file append.
  - If Read, Read full Line.

- pipe system
  - Definition
    - Make Communication Between 2 Languages or Process and Programs on same machine
    - Exchange Information, Data and Commands
    - Low latency, Low CPU load, high efficiency
  - Operation
    - Go
      - 1-go receive from UI Request
      - 2-go send Request to c++
      - 3-go wait for "done" from C++
      - 4-go read response from C++
      - 5-go send response to UI
    - C++
      - 1-C++ wait for "check" to check Request
      - 2-C++ Receive Request
      - 3-C++ Process Request
      - 4-C++ tell Go that Response is Ready
      - 5-C++ Write Response
  - Architecture
    - File Structure
      - Single file for both, open file as Reading/Writing.
      - Working On low-level IO, Using binary for streaming.
      - Contain "Flags" to manage Connection
      - Flags are 3, and they Handle and Route Connection
      - Contain "Rooms" to store Data
      - Contain Mapping System
    - Flags
      - Ready
        - Peer Can Handshake if this Ready
        - Ready for Receiving
      - Wait
        - During Process Mode
        - Peer should wait for Response
      - Done
        - Peer Can Read Response/Data
        - Cannot do Handshake until Read Response
    - Rooms
      - Room Can be Reached by Name
      - Room Can be Added,Deleted and Sized By Main Language (C++)
      - Room Store data for a while
      - Room Can be free from both Peers
      - Room is a Container for Data Exchangement
      - There are a Map for Rooms
    - Map
      - Map is a separated file
      - Map Points to Rooms
      - Room Name > Room border
      - Map is a separated tool can be used outside

- Connections
- Basic Mode
  - Normal and Simple Commands Transfaring
  - One Connection Per Operation
  - Can Transfer Only Simple Data and messages (Strings)
  - Peer to Peer Connection
  - Totally Safe, no risk
- Advanced Mode
  - Complex and need mode coding
  - Can handle Multiable Connections
  - If not Prepared Configured may cause Crash
  - Can Connect and Send to Multiable Sockets Easily
  - Can Send/Receive Large Data , Arrays, Binaries, and outside Files
  - Can be One-Shot Connection
- Protocol
  - Socket A
    - Peer ID
    - Check for
      - State if Active
      - Errors
      - Flag(Waiting Until Ready)
    - Send ID
    - Write into Pocket
    - Flip State to (READ)
    - Waiting for Seen
    - Read Connection Status
    - If Accepted, Points to Peer Socket
  - Socket B
    - Enter Listen Mode
    - Waiting for ID
    - Sleep until Read Flag
    - Read ID
    - Read Header
    - Points to Socket
    - If Accept Connection
      - Flip State to (SEEN)
      - Flip Connection Status to (ACCEPTED)

- Protocol
  - OSH
  - Sender
    - Read Connection Status
    - Check if Peer Free
    - if Free (IDLE)
    - Reserve Socket
      - Flip CS Byte into 1(RESERVED)
    - Write Header
    - Write Data
    - Wake Socket
    - Wait for (SEEN)
      - Send More Data
      - Sleep
      - End Connection
  - Receiver
    - Open Thread
    - -> Wait CS until 1
    - Listen for Incoming Sockets
    - <-
    - Flip Flag into (READ)
    - =>
    - Waiting Flag to Flip into (READ)
    - <-
    - Read Header
    - <-
    - Read Data
    - Flip Flag to (SEEN)
    - Confirm
    - Wait For Flag
    - Flip to READ
    - Flip to WAIT
    - Flip to DONE